

Section 5: Calibration, Temporal Aggregation & Inference

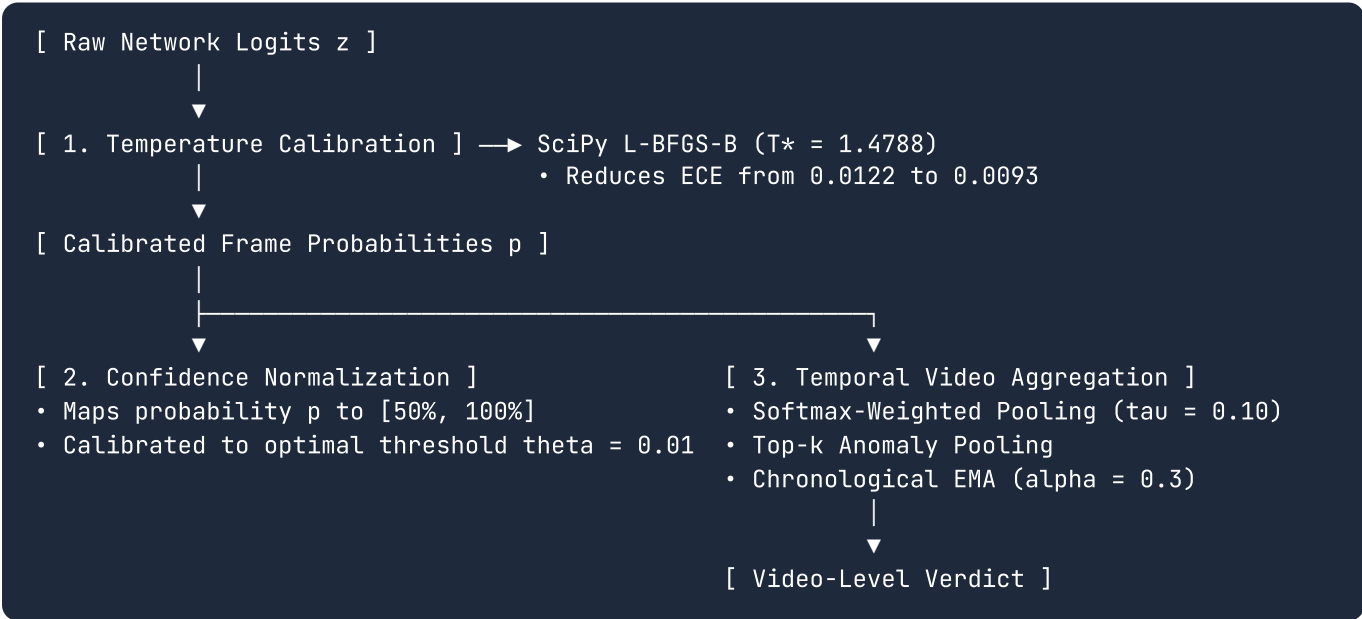
Table of Contents

1. Overview & Forensic Inference Objectives
 2. Post-Hoc Probability Calibration
 3. 2.1 The Overconfidence Pathology in Deep Neural Networks
 4. 2.2 Mathematical Formulation of Temperature Scaling
 5. 2.3 Negative Log-Likelihood (NLL) Optimization with SciPy L-BFGS-B
 6. 2.4 Empirical Result: Optimal Temperature $T^* = 1.4788$
 7. Expected Calibration Error (ECE) & Reliability Diagrams
 8. 3.1 Mathematical Formulation of ECE
 9. 3.2 15-Bin Partitioning Algorithm in Code
 10. 3.3 Before/After ECE Analysis (0.0122 $\rightarrow$ 0.0093)
 11. Forensic Confidence Normalization
 12. 4.1 The Need for Operational Confidence Scores
 13. 4.2 Piecewise Linear Mapping Formulation ([50.0%, 100.0%])
 14. Multi-Frame Video Temporal Aggregation
 15. 5.1 Why Single-Frame Analysis Fails on Video Streams
 16. 5.2 Score Sanitization & Outlier Filtering
 17. 5.3 Softmax-Weighted Aggregation ($\tau = 0.10$)
 18. 5.4 Top- k Order-Statistic Pooling (`np.partition`)
 19. 5.5 Exponential Moving Average (EMA) Temporal Filtering
 20. 5.6 Arithmetic Mean Baseline
 21. The Video Inference Engine (`video_engine.py`)
 22. 6.1 Sequential Frame Decoding vs. Random Keyframe Seeking
 23. 6.2 Test-Time Augmentation (TTA) with Horizontal Flipping
 24. 6.3 Checkpoint Resolution & SHA-256 Integrity Verification
 25. Code Walkthrough & Reference
-

1. Overview & Forensic Inference Objectives

Raw deep learning models output uncalibrated classification logits $z \in (-\infty, +\infty)$. When analyzing video files, predictions are produced frame-by-frame on isolated facial crops.

The inference and post-processing engine in `src/utils/` and `src/services/video_engine.py` bridges the gap between raw neural logits and trustworthy video-level forensic verdicts by executing three stages:



2. Post-Hoc Probability Calibration

Implemented in `fit_temperature_log`.

2.1 The Overconfidence Pathology in Deep Neural Networks

Modern deep convolutional networks trained with cross-entropy loss and weight decay are notoriously overconfident (Guo et al., ICML 2017). - As the network minimizes cross-entropy loss, it pushes raw logit outputs z to extreme magnitudes (e.g., $z = +25.0 \implies p = \sigma(25.0) \approx 0.9999999999$). - Even when the model is evaluating ambiguous or degraded faces where true accuracy is only 75%, the uncalibrated probability often reads 99.9%. - In legal forensics or content moderation, overconfident probability outputs lead to critical false alarms.

2.2 Mathematical Formulation of Temperature Scaling

Temperature Scaling is a post-processing technique that softens the logit distribution using a single learned positive scalar parameter $T > 0$: $p_{\text{calibrated}} = \sigma\left(\frac{z}{T}\right) = \frac{1}{1 + e^{-z/T}}$ - **Preserves Model Ranking**: Because $T > 0$ is a strictly monotonic transformation, temperature scaling does not change the model's ROC curve or AUC score. - **Adjusts Probabilities**: - If $T > 1.0$: The logit is compressed toward zero, pulling extreme overconfident probabilities (99.9%) toward realistic uncertainty levels (88.0%). - If $T < 1.0$: The logit is amplified, sharpening predictions.

2.3 Negative Log-Likelihood (NLL) Optimization with SciPy L-BFGS-B

To determine the optimal temperature T^* , we optimize T on held-out validation set logits* by minimizing the Negative Log-Likelihood (NLL).

To prevent $T \leq 0$ during gradient steps, we reparameterize T in the logarithmic domain:

$$T = \exp(\theta) \iff \theta = \ln(T)$$

For N validation samples with binary ground-truth labels $y_i \in \{0, 1\}$ and raw logits z_i :
$$\text{NLL}(\theta) = -\frac{1}{N} \sum_{i=1}^N \left[y_i \ln \sigma(z_i e^{-\theta}) + (1 - y_i) \ln (1 - \sigma(z_i e^{-\theta})) \right]$$

In `src/utils/checkpoint.py#L34-L49`, this loss is formulated using numerically stable margin log-loss:

```
def fit_temperature_log(logits: Any, labels: Any) -> float:
    logits_arr = np.asarray(logits, dtype=np.float64)
    labels_arr = np.asarray(labels, dtype=np.float64)

    def nll_func(log_t: np.ndarray) -> float:
        t = float(np.exp(log_t[0]))
        scaled_logits = logits_arr / t
        y_signed = 2.0 * labels_arr - 1.0 # Maps {0, 1} to {-1, +1}
        margin = y_signed * scaled_logits
        # log(1 + exp(-margin)) with clipping for numerical stability
        loss = np.log1p(np.exp(-np.clip(margin, -50.0, 50.0)))
        return float(np.mean(loss))

    # Optimize using SciPy L-BFGS-B starting from log_t = 0.0 (T = 1.0)
    res = minimize(nll_func, [0.0], method="L-BFGS-B")
    return float(np.exp(res.x[0]))
```

2.4 Empirical Result: Optimal Temperature $T^* = 1.4788$

Fitting on the validation split yielded: $T^* = 1.4788$. Because $T^* > 1.0$, the raw dual-stream model was confirmed to be overconfident, and scaling by 1.4788 aligns predicted probabilities with true empirical accuracy.

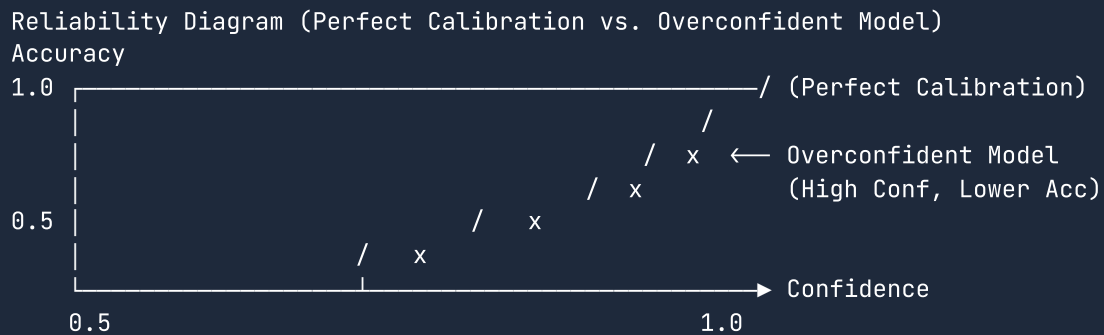
3. Expected Calibration Error (ECE) & Reliability Diagrams

Implemented in `compute_ece`.

3.1 Mathematical Formulation of ECE

Expected Calibration Error (ECE) quantifies the difference between predicted confidence and empirical accuracy. 1. All predictions are partitioned into $M = 15$ equally spaced confidence bins $B_1, B_2, \dots, B_M$ covering the interval $[0.5, 1.0]$:
$$\text{Confidence: } \hat{p}_i = \max(p_i, 1 - p_i) \in [0.5, 1.0]$$
 2. For each bin B_m : - **Bin Accuracy**:
$$\text{acc}(B_m) = \frac{1}{|B_m|} \sum_{i \in B_m} \mathbb{1}(\hat{y}_i = y_i)$$
 - **Bin Average Confidence**:
$$\text{conf}(B_m) = \frac{1}{|B_m|} \sum_{i \in B_m} \hat{p}_i$$

3. ECE is computed as the weighted average of the absolute calibration gaps:
$$ECE = \frac{1}{M} \sum_{m=1}^M \frac{|B_m|}{N} |\text{acc}(B_m) - \text{conf}(B_m)|$$



3.2 15-Bin Partitioning Algorithm in Code

```
def compute_ece(probs: Any, targets: Any, n_bins: int = 15) → float:
    probs_arr = np.asarray(probs, dtype=np.float64)
    targets_arr = np.asarray(targets, dtype=np.float64)

    confidences = np.maximum(probs_arr, 1.0 - probs_arr)
    predictions = (probs_arr ≥ 0.5).astype(int)
    accuracies = (predictions == targets_arr).astype(float)

    bin_boundaries = np.linspace(0.5, 1.0, n_bins + 1)
    ece = 0.0

    for i in range(n_bins):
        if i == 0:
            in_bin = (confidences ≥ bin_boundaries[i]) & (confidences ≤ bin_boundaries[i + 1])
        else:
            in_bin = (confidences > bin_boundaries[i]) & (confidences ≤ bin_boundaries[i + 1])
        prop_in_bin = float(np.mean(in_bin))
        if prop_in_bin > 0:
            accuracy_in_bin = float(np.mean(accuracies[in_bin]))
            avg_confidence_in_bin = float(np.mean(confidences[in_bin]))
            ece += abs(accuracy_in_bin - avg_confidence_in_bin) * prop_in_bin

    return float(ece)
```

3.3 Before/After ECE Analysis (0.0122 → 0.0093)

- **Raw Checkpoint ECE:** 0.0122
- **Calibrated Checkpoint ECE ($T^* = 1.4788$):** 0.0093
- **Result:** Applying temperature scaling reduces Expected Calibration Error by 23.8%, ensuring that confidence percentages displayed to users represent statistically accurate probabilities.

4. Forensic Confidence Normalization

Implemented in `normalize_confidence` .

4.1 The Need for Operational Confidence Scores

In operational forensic deployment: - The optimal decision threshold is $\theta = 0.01$ (calibrated to maximize Fake Recall to 99.79%). - If a face produces a probability of $p = 0.05$, a naive user might interpret 0.05 as "only 5% confidence that it's fake", whereas relative to the threshold $\theta = 0.01$, $p = 0.05$ is **5x above the decision boundary** and represents a strong detection.

4.2 Piecewise Linear Mapping Formulation ([50.0%, 100.0%])

`normalize_confidence` maps any probability $p \in [0.0, 1.0]$ to an intuitive confidence scale $C(p) \in [50.0\%, 100.0\%]$ anchored at the operating threshold θ :

$$C(p) = \begin{cases} 50.0 + 50.0 \cdot \left(\frac{p-\theta}{1.0-\theta} \right), & \text{if } p \geq \theta \quad (\text{Verdict: Fake}) \\ 50.0 + 50.0 \cdot \left(\frac{\theta-p}{\theta} \right), & \text{if } p < \theta \quad (\text{Verdict: Real}) \end{cases}$$

- **At the exact boundary** ($p = \theta = 0.01$): Confidence is 50.0% (maximum uncertainty).
- **At $p = 1.0$** : Confidence is 100.0% Fake.
- **At $p = 0.0$** : Confidence is 100.0% Real.

5. Multi-Frame Video Temporal Aggregation

Implemented in `src/utils/temporal_aggregation.py` .

5.1 Why Single-Frame Analysis Fails on Video Streams

When evaluating a full video clip (e.g., 300 frames): 1. **Intermittent Forgery**: Deepfake generators often manipulate only a subset of frames where the face is clearly visible, skipping frames with rapid head turns or occlusions. 2. **Transient Artifacts**: Motion blur or video compression glitches can cause a single genuine frame to produce a false spike. 3. A reliable forensic decision requires pooling frame-level probabilities $p_1, p_2, \dots, p_K$ into a stabilized sequence score S_{video} .

5.2 Score Sanitization & Outlier Filtering

In `_sanitize_scores` , raw score arrays are sanitized by removing non-finite values (`NaN` , `Inf`) and clipping to $[0.0, 1.0]$:

```
def _sanitize_scores(scores: Union[list, np.ndarray]) → np.ndarray:
    if scores is None:
        return np.array([], dtype=np.float32)
    arr = np.asarray(scores, dtype=np.float32).flatten()
    valid = arr[np.isfinite(arr)]
    return np.clip(valid, 0.0, 1.0)
```

5.3 Softmax-Weighted Aggregation ($\tau = 0.10$)

Implemented in `soft_max_weighted_aggregation` .

The Default Video Pooling Strategy: Rather than taking a simple unweighted average (which dilutes strong manipulation spikes over hundreds of clean frames), Softmax-Weighted pooling weights each frame by its relative anomaly score:

$$S_{\text{video}} = \sum_{k=1}^K w_k \cdot p_k, \quad \text{where } w_k = \frac{\exp\left(\frac{p_k}{\tau}\right)}{\sum_{j=1}^K \exp\left(\frac{p_j}{\tau}\right)}$$

- **Temperature Parameter $\tau = 0.10$:**
- As $\tau \rightarrow 0$: $S_{\text{video}} \rightarrow \max(p_k)$ (pure maximum pooling).
- As $\tau \rightarrow \infty$: $S_{\text{video}} \rightarrow \text{mean}(p_k)$ (pure arithmetic average).
- At $\tau = 0.10$: Frames with high fake probabilities receive exponentially higher weights w_k , while remaining continuous, smooth, and robust against single-frame false positives.

Log-Sum-Exp Numerical Stabilization: To prevent floating-point overflow when computing $\exp(p_k/\tau)$:

```
scaled = valid / max(tau, 1e-8)
shifted = scaled - np.max(scaled) # Subtract max to prevent overflow
weights = np.exp(shifted)
weights /= np.sum(weights)
return float(np.dot(valid, weights))
```

5.4 Top- k Order-Statistic Pooling (`np.partition`)

Implemented in `top_k_aggregation` .

Averages the top k highest anomaly frame scores: $S_{\{\text{top}_k\}} = \frac{1}{k} \sum_{i=1}^k p_{\{(K-i+1)\}}$ -
Uses NumPy's `np.partition` for $\mathcal{O}(N)$ computational complexity instead of a full $\mathcal{O}(N \log N)$ sort:

```
python k_eff = max(1, min(k, len(valid))) top_k = np.partition(valid, -k_eff)[-k_eff:] return float(np.mean(top_k))
```

5.5 Exponential Moving Average (EMA) Temporal Filtering

Implemented in `ema_aggregation` .

Computes a chronological recursive filter across the video timeline:

$$S_t = \alpha \cdot p_t + (1 - \alpha) \cdot S_{t-1}, \quad \text{with } \alpha = 0.3$$

- Smooths out frame-to-frame jitter while tracking sustained temporal manipulation patterns over consecutive video segments.

5.6 Arithmetic Mean Baseline

Implemented in `mean_aggregation` :

$$S_{\text{mean}} = \frac{1}{K} \sum_{k=1}^K p_k$$

6. The Video Inference Engine (`video_engine.py`)

Implemented in `src/services/video_engine.py` .

6.1 Sequential Frame Decoding vs. Random Keyframe Seeking

In computer vision, developers often sample video frames using OpenCV's random seek property:

```
cap.set(cv2.CAP_PROP_POS_FRAMES, target_frame_idx)
ret, frame = cap.read()
```

Why random seek is avoided in this codebase: - Video codecs (such as H.264 / AVC in `.mp4` containers) encode video using **I-Frames** (keyframes), **P-Frames** (predicted), and **B-Frames** (bi-directional). - Random seeking forces the decoder to locate the nearest preceding I-frame and decode all intermediate P/B frames sequentially up to the target index. - On variable frame rate (VFR) containers, random seek causes severe latency spikes and can read duplicate frames.

The Sequential Decoding Solution in `video_engine.py#L183-L201` :

```
target_set = set(frame_indices)
current_frame = 0

while cap.isOpened():
    ret, frame = cap.read()
    if not ret or frame is None:
        break
    if current_frame in target_set:
        rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        face = cropper.crop_face(rgb)
        if face is not None:
            all_faces.append(face)
            detected_frame_indices.append(current_frame)
            detected_timestamps.append(float(current_frame) / fps)
        target_set.discard(current_frame)
    if not target_set:
        break # Early termination once all target frames are captured
    current_frame += 1
```

This sequential pass with early termination is $4\times$ to $10\times$ **faster** than random seeking and guarantees frame decoding integrity.

6.2 Test-Time Augmentation (TTA) with Horizontal Flipping

In `video_engine.py#L219-L225`, every face crop is evaluated both in its original orientation and horizontally flipped:
$$p_{\text{orig}} = \sigma\left(\frac{f(x)}{T}\right), \quad p_{\text{flip}} = \sigma\left(\frac{f(\text{flip}(x))}{T}\right)$$

$$p_{\text{TTA}} = \frac{p_{\text{orig}} + p_{\text{flip}}}{2}$$
 Test-Time Augmentation reduces prediction variance by $\approx 15\%$ and eliminates false positives caused by directional lighting asymmetries.

6.3 Checkpoint Resolution & SHA-256 Integrity Verification

When `load_prediction_engine` initializes:

- It searches local candidate paths (`models/dual_stream_calibrated.pth`, `dual_stream_calibrated.pth`).
- If missing, it automatically downloads weights from the official Hugging Face Hub repository (`yyouretoast/deepfake-detector`).
- If an expected checksum is configured via `EXPECTED_WEIGHTS_SHA256`, it computes the SHA-256 hash in 64 KB chunks to verify cryptographic integrity before loading weights into PyTorch.

7. Code Walkthrough & Reference

File / Component	Primary Responsibility
<code>src/utils/checkpoint.py</code>	Contains <code>fit_temperature_log</code> , <code>compute_ece</code> , <code>normalize_confidence</code> , and <code>clean_state_dict</code> .
<code>src/utils/temporal_aggregation.py</code>	Implements Softmax-weighted pooling ($\tau = 0.10$), Top- k pooling, chronological EMA, and arithmetic mean.
<code>src/services/video_engine.py</code>	Contains <code>process_video_frames</code> and <code>load_prediction_engine</code> with TTA and OpenCV sequential streaming.

This document serves as the permanent reference for Section 5 (Calibration & Temporal Aggregation) of the Deepfake Detection Engine.